Unidad 4 / Semana 9
Licenciatura en Ciberdefensa

---

1. Concepto de Interbloqueo

1.1 Definición

Un **interbloqueo** (o *deadlock*) ocurre cuando dos o más procesos quedan bloqueados indefinidamente porque cada uno espera un recurso que otro proceso del conjunto retiene. Es un problema crítico en sistemas concurrentes.

Ejemplo clásico:

- Proceso A retiene la **impresora** y solicita el **escáner**.
- Proceso B retiene el **escáner** y solicita la **impresora**.
  Resultado: Ambos procesos se bloquean mutuamente.

1.2 Condiciones Necesarias (Coffman, 1971)

Para que ocurra un interbloqueo, deben cumplirse **cuatro condiciones simultáneamente**:

Condición	Explicación	Ejemplo
Exclusión mutua	Solo un proceso puede usar un recurso a la vez.	Impresora (no compartible).
Retención y espera	Procesos retienen recursos mientras esperan otros nuevos.	A retiene CPU y espera memoria.
No expropiación	El SO no puede quitar recursos asignados a un proceso.	Archivos abiertos.
Espera circular	Cadena circular de procesos donde cada uno espera un recurso del siguiente.	A → B → C → A.

Importante: Si se elimina *al menos una* de estas condiciones, se evita el deadlock.

Recursos Apropiativos vs No Apropiativos en Sistemas Operativos

Definición y Características

Recursos No Apropiativos

- **Definición**: Recursos que **no pueden** ser retirados a un proceso sin causar fallos o inconsistencias
- **Características**:
  - Una vez asignados, deben ser liberados voluntariamente por el proceso
  - El SO no puede forzar su liberación

    - Son los principales causantes de interbloqueos
- **Ejemplos**:
    - Impresoras
    - Archivos abiertos en modo escritura
    - Dispositivos de E/S (scanners)
    - Semáforos y candados

Recursos Apropiativos

- **Definición**: Recursos que **pueden** ser retirados a un proceso y reasignados a otro
- **Características**:
    - El SO puede gestionarlos dinámicamente
    - Su expropiación no causa problemas de consistencia
    - Permiten mayor flexibilidad en la gestión de recursos
- **Ejemplos**:
    - CPU (mediante el scheduler)
    - Memoria RAM
    - Algunos dispositivos de almacenamiento

Comparativa Detallada

Característica	Recurso No Apropiativo	Recurso Apropiativo
Liberación	Solo por el proceso dueño	Puede ser expropiado por SO
Riesgo Deadlock	Alto	Bajo o nulo
Ejemplos típicos	Impresoras, archivos	CPU, memoria RAM
Gestión por el SO	Limitada	Total
Velocidad acceso	Generalmente lenta	Generalmente rápida

Implicaciones en Deadlocks

Para recursos no apropiativos

1. Son la principal causa de interbloqueos
2. Requieren estrategias especiales de prevención:
    - Asignación jerárquica
    - Solicitud de todos los recursos al inicio
    - Protocolos de evitación

Para recursos apropiativos

1. El SO puede resolver deadlocks expropiándolos
2. Estrategias comunes:
    - Planificación por prioridades
    - Time-sharing
    - Reasignación dinámica

Ejemplo Práctico

Caso de interbloqueo:

1. Proceso A tiene:
    - Recurso no apropiativo: Impresora
    - Solicita: Scanner (otro recurso no apropiativo)
2. Proceso B tiene:
    - Recurso no apropiativo: Scanner
    - Solicita: Impresora

Solución posible:

- Si alguno fuera apropiativo (ej: memoria), el SO podría:
    1. Expropiar la memoria a proceso A
    2. Asignársela a proceso B
    3. Permitir que B complete
    4. Liberar recursos
    5. Reasignar a A

Técnicas de Gestión

Para no apropiativos

- **Algoritmo del banquero**: Previene deadlocks verificando estados seguros
- **Órdenes jerárquicos**: Asignar recursos en orden predefinido
- **Timeouts**: Liberar recursos tras un tiempo de espera

Para apropiativos

- **Planificación Round Robin**: Asigna tiempos de CPU
- **Swapping**: Manejo dinámico de memoria
- **Prioridades**: Asignación basada en urgencia

Conclusión

La distinción entre recursos apropiativos y no apropiativos es fundamental para:

- Diseñar sistemas operativos eficientes
- Implementar estrategias de prevención de deadlocks
- Optimizar la gestión de recursos del sistema

2. Estrategias para Manejar Interbloqueos

2.1 Prevención

Objetivo: Eliminar una o más condiciones de Coffman.

- **Exclusión mutua**:

- *Solución*: Usar recursos compartibles (ej: memoria de solo lectura).
- *Limitación*: No aplicable a recursos críticos (ej: impresora).

- **Retención y espera**:
  - *Solución*: Solicitar **todos los recursos necesarios al inicio** (poco eficiente).
  - *Alternativa*: Liberar recursos asignados si no se obtienen los nuevos.

- **No expropiación**:
  - *Solución*: Permitir al SO expropiar recursos (solo aplicable a CPU/memoria).

- **Espera circular**:
  - *Solución*: **Ordenar recursos jerárquicamente** (ej: numerar discos, impresoras, etc.).
  - *Regla*: Los procesos deben solicitar recursos en orden ascendente.

2.2 Predicción (Algoritmo del Banquero)

Funcionamiento:

1. El SO simula asignaciones futuras.
2. Verifica si el sistema entraría en un **estado seguro** (donde todos los procesos pueden terminar).
3. Rechaza solicitudes que lleven a estados inseguros.

Limitaciones:

- Requiere conocimiento previo de la demanda máxima de recursos.
- Costoso computacionalmente.

2.3 Detección

Métodos:

- **Grafos de asignación**: Buscar ciclos en el grafo de recursos.
- **Matrices de recursos**: Comparar recursos disponibles con las demandas pendientes.

Ejemplo de matriz:

Proceso	Recurso 1 (Asignado)	Recurso 2 (Solicitado)
A	1	0
B	0	1

2.4 Omisión ("Algoritmo del Avestruz")

- **Usado en sistemas como Linux/Windows**.
- Se ignora el problema y se reinician procesos si ocurre un deadlock.
- **Justificación**: El costo de prevención/detección supera los beneficios en sistemas generales.

---

3. Recuperación de Interbloqueos

3.1 Abortar Procesos

- **Selección de víctimas**: Basada en:
  - Prioridad del proceso.
  - Tiempo de ejecución (evitar perder trabajo largo).
  - Recursos utilizados.

3.2 Expropiación

- Solo aplicable a recursos **apropiativos** (CPU, memoria).
- **Problema**: Puede causar inanición (starvation).

3.3 Rollback

- Revertir procesos a un **punto de control seguro** (*checkpoint*).
- Complejidad alta: Requiere guardar estados periódicamente.

Ejemplos Prácticos

Caso 1: Base de Datos

- **Deadlock**: Transacción T1 bloquea fila X y espera Y; T2 bloquea Y y espera X.
- **Solución**: El gestor de bases de datos aborta T1 o T2 tras timeout.

Caso 2: Redes

- **Deadlock en routers**: Paquetes circulares entre nodos.
- **Solución**: Protocolos como **Spanning Tree** (IEEE 802.1D).

Interbloqueos (Deadlocks) en Ciberdefensa: Relevancia y Aplicaciones

1. Importancia en Seguridad de Sistemas

1.1 Impacto en Infraestructuras Críticas

- **Ataques por denegación de servicio (DoS)**:
  - Técnica: Inducir deadlocks artificiales en sistemas SCADA/OT
  - Objetivo: Paralizar operaciones en plantas energéticas/hídricas
  - Caso real: Stuxnet (2010) afectó PLCs mediante bloqueos de recursos
- **Secuestro de recursos**:
  - Método: Acaparamiento deliberado de recursos no apropiativos
  - Ejemplo: Bloqueo de dispositivos IoT mediante saturación de semáforos

1.2 Mecanismos de Defensa

Técnica	Aplicación en Ciberdefensa	Ejemplo Implementación
Detección temprana	Monitoreo de grafos de asignación	SIEMs con patrones de deadlock
Microsegmentación	Aislamiento de recursos críticos	Kubernetes con límites de recursos
Priorización	Schemas QoS para tráfico esencial	VLANs para sistemas de mando/control

2. Aplicaciones en Criptografía y Blockchain

2.1 Smart Contracts

- **Riesgo**: Deadlocks en contratos autoejecutables
    - Ejemplo: Dos contratos que se esperan mutuamente en Ethereum
    - Solución: Patrón "Checks-Effects-Interactions"
- **Técnicas preventivas**:

```
// Ejemplo de código seguro
function transfer(address to, uint amount) external {
    require(balances[msg.sender] >= amount); // CHECK
    balances[msg.sender] -= amount;          // EFFECT
    (bool success,) = to.call{value: amount}(""); // INTERACTION
    require(success);
}
```

2.2 Libros Distribuidos

- Problema: Consenso en redes blockchain
- Algoritmos PoW/PoS como mecanismos anti-deadlock
- Caso: Bitcoin usa tiempo de bloque como recurso apropiativo

3. Tácticas Ofensivas (Red Team)

3.1 Explotación de Vulnerabilidades

- Técnicas comunes:
    1. Resource Starvation:
        - Saturation de file descriptors
        - Ejemplo: ulimit manipulation en Linux
    2. Lock Contention:
        - Ataque a mutex en servicios web (ej. Apache)
- Herramientas:

```
# Simulador de deadlock en Python
```

```
import threading
lock1 = threading.Lock()
lock2 = threading.Lock()

def thread1():
    with lock1:
        with lock2: pass

def thread2():
    with lock2:
        with lock1: pass  # Deadlock intencional
```

3.2 Persistencia en Sistemas

- Técnica: Inyección de deadlocks persistentes
  - Modificación de parámetros de scheduler
  - Corrupción de tablas de recursos del kernel

4. Estrategias Defensivas (Blue Team)

4.1 Hardening de Sistemas

- Configuraciones clave:

```
# /etc/security/limits.conf
* hard nproc 500      # Limita procesos por usuario
* hard nofile 1024    # Restringe file descriptors
```

- Buenas prácticas:
  - Uso de timeouts en todas las operaciones de bloqueo
  - Implementación de circuit breakers

4.2 Detección Avanzada

- Técnicas:
  - Análisis estático: Detección de patrones en código

```
// Patrón riesgoso
synchronized(res1) {
synchronized(res2) { ... }  // Potencial deadlock
}
```

- Machine Learning: Modelos predictivos basados en telemetría

5. Caso de Estudio: Sistemas de Defensa Cibernética

5.1 Sistemas IDS/IPS

- Problema: Deadlocks en análisis de paquetes paralelos
- Solución: Arquitectura Reactiva con backpressure

5.2 Plataformas de Threat Intelligence

- Desafío: Bloqueos en bases de datos de IOCs
- Implementación: Uso de Redis con expiración automática

6. Futuro en Ciberdefensa

6.1 Tendencias Emergentes

- Quantum Computing:
  - Deadlocks en qubits (problema de decoherencia)
  - Soluciones con algoritmos QLocks
- Edge Computing:
  - Protocolos anti-deadlock para dispositivos IoT
  - Ejemplo: Time-Slotted Channel Hopping (TSCH)

6.2 Investigación Actual

- Tópicos relevantes:
  - Formal verification de sistemas libres de deadlock
  - Hardware transactional memory para ciberseguridad

🧠 Unidad 4 – Semana 10

💻 Sistemas Operativos II

Tema: Interbloqueos y el Algoritmo del Banquero

◇ 1. Introducción

En los **sistemas operativos**, un **interbloqueo (deadlock)** ocurre cuando varios procesos quedan **bloqueados indefinidamente** al esperar recursos que otros procesos no liberan.

Existen **cuatro estrategias principales** para manejar los interbloqueos:

Estrategia	Descripción	Ejemplo
Prevención	Evita que se cumplan las condiciones que producen el interbloqueo.	No permitir que un proceso retenga un recurso mientras espera otro.

Estrategia	Descripción	Ejemplo
Evitación / Predicción	Analiza dinámicamente las solicitudes de recursos para mantener el sistema en estado **seguro**.	**Algoritmo del Banquero.**
Detección	Permite el interbloqueo pero lo **detecta** mediante algoritmos.	Escaneo periódico de procesos bloqueados.
Recuperación	Se actúa **una vez detectado** el interbloqueo (reiniciar procesos, liberar recursos).	Terminar procesos para romper el ciclo.

> 🛠️ También existe el **algoritmo del avestruz**, que **ignora el problema** hasta que se interviene manualmente.

◇ 1.1 El Algoritmo del Banquero

Creado por **Edsger W. Dijkstra (1965)** en el contexto del **Sistema Operativo THE**.

🏦 Analogía del "Banquero"

- El **banco** tiene una cantidad limitada de dinero (recursos).
- Los **clientes** (procesos) solicitan préstamos (recursos).
- El **banquero** concede préstamos solo si puede **garantizar que todos los clientes podrán devolver el dinero**, es decir, si el sistema puede mantenerse **en un estado seguro**.

El objetivo es **evitar entrar en un estado inseguro**, donde podría producirse un **interbloqueo**.

◇ 2. Funcionamiento del Algoritmo del Banquero

2.1 Conceptos Clave

Concepto	Definición
Estado del sistema	Representa los recursos asignados y disponibles en un momento dado.
Estado seguro	Existe al menos una secuencia de ejecución en la que todos los procesos pueden finalizar.
Estado inseguro	No se puede garantizar que todos los procesos finalicen; puede llevar a interbloqueo.
Secuencia segura	Orden de ejecución de procesos sin bloqueos.
Proceso	Programa que solicita recursos para ejecutarse.
Recurso	Elemento como CPU, memoria, archivos, etc.
Available	Vector con los recursos disponibles actualmente.
Max	Matriz con la demanda máxima de recursos que podría hacer cada proceso.

Concepto	Definición
Allocation	Matriz con los recursos actualmente asignados a cada proceso.
Need	Matriz con los recursos que aún necesita cada proceso para completar: **Need = Max – Allocation**

◇ Simulación de Asignación

Antes de conceder recursos, el sistema **simula la asignación**:

- Si el sistema **permanece en estado seguro**, se concede.
- Si el sistema pasa a **estado inseguro**, se **rechaza temporalmente**.

◇ 2.1 Pasos del Algoritmo del Banquero

1. **Declaración inicial:**
   Cada proceso declara su demanda máxima (**Max**).
   El sistema conoce los recursos disponibles (**Available**) y los asignados (**Allocation**).

2. **Cálculo de necesidades:**
   `Need = Max - Allocation`

3. **Solicitud de recursos:**
   El proceso pide recursos (Request).
   Se verifica:

   - Que `Request ≤ Need`
   - Que `Request ≤ Available`

4. **Simulación:**
   Se simula la asignación temporal:

   - `Available = Available - Request`
   - `Allocation = Allocation + Request`
   - `Need = Need - Request`

5. **Verificación de estado seguro:**

   - Se busca una **secuencia segura** de ejecución.
   - Si todos los procesos pueden finalizar → Estado **seguro**.
   - Si no → Estado **inseguro**.

6. **Decisión:**

   - **Seguro:** se concede la solicitud.
   - **Inseguro:** se rechaza temporalmente.

◇ 2.2 Ejemplos Prácticos

🟢 Ejemplo 1: Estado SEGURO

Total de recursos:

`[A=10, B=5, C=7]`

Max

Proceso	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2

Allocation

Proceso	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1

Cálculo de Available

```
A: 10 - (0+2+3+2) = 3
B:  5 - (1+0+0+1) = 3
C:  7 - (0+0+2+1) = 4
→ Available = [3,3,4]
```

Cálculo de Need

Proceso	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1

Simulación de ejecución:

1. P1 puede ejecutarse (`Need ≤ Available`).
   → Libera sus recursos `[2,0,0]`.
   → Available = `[5,3,4]`.

2. P3 puede ejecutarse (`[0,1,1] ≤ [5,3,4]`).
   → Libera `[2,1,1]`.
   → Available = `[7,4,5]`.

3. P0 puede ejecutarse (`[7,4,3] ≤ [7,4,5]`).
   → Libera `[0,1,0]`.
   → Available = `[7,5,5]`.

4. P2 puede ejecutarse (`[6,0,0] ≤ [7,5,5]`).
   → Libera `[3,0,2]`.
   → Available = `[10,5,7]`.

☑ Todos los procesos finalizan → **Estado SEGURO.**

🟤 Ejemplo 2: Estado INSEGURO

Allocations

Proceso	A	B	C
P0	3	1	0
P1	2	1	1
P2	3	0	2
P3	2	3	1

Cálculo de Availables

```
A: 10 - 10 = 0
B: 5 - 5 = 0
C: 7 - 4 = 3
→ Available = [0,0,3]
```

Need

Proceso	A	B	C
P0	4	4	3
P1	1	1	1
P2	6	0	0
P3	0	1	1

Ningún proceso puede ejecutarse, ya que todos necesitan más recursos que los disponibles.

🚫 **Estado INSEGURO** → riesgo de interbloqueo.

🧩 Ejemplo comentado (interpretación práctica)

Imaginemos un **sistema militar de ciberdefensa**:

- Los recursos son **procesadores, memoria y canales de red**.
- Los procesos son **análisis forense, detección de intrusos, cifrado de datos**, etc.

El **algoritmo del banquero** evita que el sistema conceda tantos recursos a tareas parciales que impidan terminar las tareas críticas.

> 💡 En la práctica, este algoritmo **no se usa directamente**, pero es **fundamental para entender la gestión segura de recursos** en entornos concurrentes.

📓 Conclusión

El **Algoritmo del Banquero** es un **método de evitación de interbloqueos** que:

- Garantiza que el sistema permanezca en un **estado seguro**.
- Simula la asignación de recursos antes de concederlos.
- Se basa en una lógica preventiva similar a la de un banco que **no presta más dinero del que puede recuperar**.

Aunque **no se usa actualmente** en los sistemas operativos, **su valor didáctico** es esencial para comprender la **gestión de concurrencia y recursos compartidos**.

🧠 Resumen Unidad 5 – Semana 11: Sistemas Distribuidos

📓 Introducción

La unidad aborda la **evolución de los sistemas operativos centralizados hacia los distribuidos**, destacando cómo estos últimos solucionan las limitaciones de escalabilidad, colaboración y eficiencia de los primeros.

- **Sistemas centralizados:** operan sobre una sola máquina con todos los recursos locales. Los usuarios pueden tener terminales remotas, pero no colaboran activamente. Difíciles de escalar.
- **Sistemas distribuidos:** surgen para coordinar múltiples nodos conectados. Permiten colaboración, tolerancia a fallos, escalabilidad y flexibilidad.
- Ejemplos modernos: **Kubernetes, Hadoop, sistemas en la nube**, etc.

💻 1. Sistemas Distribuidos

Un **Sistema Distribuido (SD)** es un conjunto de componentes de hardware y software en distintas computadoras conectadas en red que actúan coordinadamente como un sistema único.

Desde la perspectiva del usuario, el sistema parece una sola entidad.

🔧 Componentes y principios

- Los componentes **se comunican y coordinan mediante mensajes**.
- Cada componente es **autónomo, modular** y tiene **interfaces definidas**.
- Permite **compartir recursos** eficientemente y **especializar funciones** (por ejemplo, un SGBD dedicado a la gestión de datos).

🎯 Objetivos principales

1. **Acceso y compartición de recursos** remotos de forma eficiente.
2. **Transparencia:**
    - **De acceso:** oculta el método de acceso a recursos.
    - **De ubicación:** el usuario no sabe dónde están físicamente.
    - **De concurrencia:** varios usuarios usan el mismo recurso sin interferir.
    - **De migración:** los recursos pueden moverse sin afectar su uso.
3. **Fiabilidad:** tolerancia a fallos mediante replicación.
4. **Rendimiento:** ejecución paralela en múltiples CPUs.

🧩 1.1 Tipos de Sistemas Distribuidos

Tipo	Descripción	Ejemplo
Sistemas de Redes de Computadoras	Cada equipo tiene su propio SO, y la red permite comunicación y acceso a servicios remotos.	Servidor de archivos en red.
Sistemas Multiprocesadores	Varios procesadores comparten una memoria principal bajo un único SO. Comunicación muy rápida.	Computadoras con múltiples CPUs.

☑ Ventajas y ❌ Desventajas

Ventajas	Desventajas
Mayor fiabilidad	Mayor complejidad de diseño
Mejor rendimiento	Más vulnerabilidades de seguridad
Escalabilidad	Mayor costo de hardware/software
Compartir recursos	Latencia de red
Comunicación eficiente	Dificultad en gestión y actualización de software

---

🏗 1.2 Arquitecturas Distribuidas

Las **Arquitecturas Distribuidas (AD)** son un conjunto de principios y guías para diseñar sistemas distribuidos eficientes y coherentes.

Objetivos

1. **Simplificar el diseño:** estructura unificada que facilita desarrollo y mantenimiento.
2. **Optimizar costos:** diseño modular y reutilizable.
3. **Fomentar la reutilización:** componentes autónomos y estandarizados.

🖧 1.2.1 Arquitectura Cliente-Servidor

Modelo de diseño donde las tareas se dividen entre:

- **Clientes:** solicitan servicios.
- **Servidores:** los proveen.

Ambos son software que pueden ejecutarse en diferentes dispositivos.

🔑 Conceptos clave

- **Cliente:** inicia solicitudes de servicio.
- **Servidor:** escucha y responde.
- **Comunicación:** se da a través de la red mediante protocolos (ej. HTTP).
- **Petición/Respuesta:** ciclo básico de operación.
- **Inspector:** herramienta para depurar la comunicación cliente-servidor.

⚙️ Funcionamiento básico

1. El cliente inicia la comunicación.
2. Envía una solicitud al servidor.
3. El servidor procesa y genera una respuesta.
4. Envía la respuesta al cliente.
5. El cliente la muestra al usuario.

🧿 Características

- **Separación de responsabilidades:** cliente → interfaz / servidor → lógica y datos.
- **Centralización de datos:** gestión de seguridad y consistencia.
- **Escalabilidad:** pueden añadirse más servidores.
- **Interoperabilidad:** distintos lenguajes o sistemas operativos mediante protocolos comunes.

Ejemplo: un **navegador web** (cliente) accediendo a un **servidor web**.

Roles

Cliente:

- Interactivo, ejecutado localmente.
- Inicia conexiones y finaliza al terminar su tarea.

Servidor:

- No interactivo, en segundo plano.
- Se ejecuta desde el arranque (daemon).
- Atiende peticiones concurrentes.

🪜 Capas lógicas

Se distinguen tres niveles:

1. **Interfaz de usuario.**
2. **Procesamiento (lógica de negocio).**
3. **Datos.**

🌐 1.2.2 Modelo Peer-to-Peer (P2P)

En este modelo, **todos los nodos tienen igual jerarquía** (no hay clientes ni servidores dedicados).
Cada nodo puede actuar como proveedor y consumidor de recursos.

🎯 Objetivos

- Compartir datos y recursos a gran escala.
- Evitar servidores dedicados: usa la capacidad de las computadoras conectadas.

💡 Características

- **Descentralización y autoorganización.**
- **Equilibrio dinámico de carga** entre nodos.
- **Contribución universal** de recursos.
- **Igualdad funcional** entre participantes.
- **Autonomía y resiliencia.**
- **Anonimato opcional.**
- **Alta eficiencia algorítmica** en búsqueda y distribución.

☑️ Ventajas

- **Escalabilidad:** más nodos = mejor rendimiento.
- **Eficiencia:** uso óptimo del ancho de banda.
- **Resiliencia:** sin punto único de fallo.
- **Flexibilidad:** nodos pueden unirse o salir sin afectar la red.
- **Privacidad y anonimato.**

❌ Desventajas

- **Seguridad débil:** sin autoridad central.
- **Riesgo legal:** intercambio de contenido ilegal.
- **Calidad variable del servicio:** depende de los nodos conectados.

🧭 Aplicaciones

- **Compartición de archivos:** BitTorrent.
- **CDN descentralizadas:** mejora tiempos de entrega.
- **Criptomonedas:** Bitcoin (libro mayor distribuido).
- **Entretenimiento:** Spotify (streaming).
- **Videojuegos:** conexión directa entre jugadores.
- **Salud:** procesamiento de datos médicos distribuidos.
- **Telecomunicaciones:** Skype (comunicación directa).

🏛 1.2.3 Arquitecturas Multi-Nivel (N-Tier)

Separan la aplicación en **niveles lógicos y físicos**:

- **Capa (Layer):** separación lógica del código (presentación, negocio, datos).
- **Nivel (Tier):** separación física, dónde se ejecuta cada capa (cliente, servidor, base de datos).

🔢 Modelo clásico: Arquitectura de 3 niveles

Nivel	Función	Ubicación	Ejemplo
1. Presentación (IU)	Interfaz con el usuario	Cliente (navegador o app)	Página web
2. Lógica de Negocio	Procesamiento, reglas, validación	Servidor de aplicación	Middleware o backend
3. Datos	Almacenamiento y acceso	Servidor de base de datos	MySQL, MongoDB

💼 Ventajas

- **Escalabilidad independiente** por nivel.
- **Reutilización de lógica de negocio** para múltiples interfaces.
- **Mantenimiento modular.**
- **Seguridad:** separación física del nivel de datos.
- **Balanceo de carga:** distribución entre múltiples servidores.

🧩 Conclusión general

Los **sistemas operativos distribuidos** representan una evolución natural hacia entornos **colaborativos, escalables y tolerantes a fallos**.
Comprender sus **modelos arquitectónicos (cliente-servidor, P2P, multinivel)** permite diseñar sistemas eficientes, seguros y adaptables a las necesidades actuales como **la nube, la web moderna y las redes descentralizadas**.

🧠 Resumen Completo – Unidad 5 / Semana 11 – Sistemas Operativos II

◇ Introducción

El **software de red** permite la **comunicación entre equipos** ubicados en redes diferentes (comunicación extremo a extremo).
Su función es gestionar todos los aspectos de la transmisión de información:

- **Eléctricos:** cables, señales, conectores.
- **Lógicos:** agrupación de bits, control de errores.
- **De direccionamiento:** identificación y envío correcto del mensaje.

Para simplificar esta complejidad, se desarrollaron **modelos por capas**, donde cada nivel cumple una función específica.

🌐 Antecedentes históricos

En los años 70 surgieron tres hitos clave:

1. **Ethernet** (para redes locales).
2. **Sistema operativo UNIX**, independiente de marcas.
3. **Protocolos TCP/IP**, base de Internet.

La **ISO** y el **CCITT** diseñaron el **modelo OSI** para estandarizar y estructurar las comunicaciones.

◇ 1. Jerarquía de protocolos

Las redes se organizan en **capas o niveles**, cada una brinda **servicios** a la superior mediante **protocolos**, que son reglas de comunicación.

- Los protocolos actúan como "**idiomas**" entre dispositivos.
- Ejemplo:
  - **HTTP** → comunicación entre navegador y servidor web.
  - **FTP** → transferencia de archivos.

👉 **Servicios**: lo que una capa ofrece.
👉 **Protocolos**: cómo se implementa ese servicio.

◇ 2. Arquitectura de red

La **división en capas** facilita la modularidad y la interoperabilidad.

Características:

- Cada capa resuelve un problema específico.
- Las capas se apoyan unas sobre otras.
- Cambiar una capa no afecta al resto.

Objetivos:

- Conectividad
- Modularidad

- Facilidad de uso, modificación e implementación
- Confiabilidad
- Interoperabilidad (equipos de distintos fabricantes funcionan juntos)

◇ 3. Modelo de referencia OSI (Open Systems Interconnection)

Modelo teórico que **divide la comunicación en 7 capas**, cada una con funciones y protocolos específicos.

🛠️ Capas del modelo OSI

1. **Física:** Transmisión de bits.
2. **Enlace de datos:** Control de errores y tramas.
3. **Red:** Direccionamiento y encaminamiento (IP).
4. **Transporte:** Comunicación extremo a extremo (TCP/UDP).
5. **Sesión:** Establece, mantiene y finaliza sesiones.
6. **Presentación:** Traducción, cifrado, compresión.
7. **Aplicación:** Interacción con el usuario (HTTP, FTP, etc.).

Servicios vs Protocolos:

- **Servicio:** qué hace una capa.
- **Protocolo:** cómo lo hace.

◇ 4. Modelo TCP/IP

🕰️ Historia

Creado por el **Departamento de Defensa de EE. UU. (ARPA)** para garantizar comunicación resistente ante fallas (ARPANET, 1969).
En 1974, **Cerf y Kahn** publican los protocolos **TCP** y **IP**.

🧱 Capas del modelo TCP/IP

1. **Acceso a red (enlace)**
2. **Interred**
3. **Transporte**
4. **Aplicación**

📦 PDU (Protocol Data Units)

Capa	Unidad de datos	Ejemplo
Aplicación	Datos	HTTP request
Transporte	Segmento	TCP
Interred	Paquete	IP
Enlace	Trama	Ethernet

Capa	Unidad de datos	Ejemplo
Física	Bits	Señales eléctricas

◇ 5. Capa de interred (nivel de red)

Gestiona el **encaminamiento de datos entre redes**.
Permite la interconexión global de redes (Internet).

5.1 Direcciones IPv4

- **32 bits** → 4 octetos (0–255).
- Identifican de forma única cada dispositivo.

Tipos:

- Pública
- Privada
- Estática
- Dinámica

Clases de direcciones

Clase	Estructura	Redes	Hosts	Rango	Máscara
A	r.h.h.h	128	16.777.214	0.0.0.0–127.0.0.0	255.0.0.0
B	r.r.h.h	16.384	65.534	128.0.0.0–191.255.0.0	255.255.0.0
C	r.r.r.h	2.097.152	254	192.0.0.0–223.255.255.0	255.255.255.0
D	Multicast	–	–	224.0.0.0–239.255.255.255	–
E	Reservada	–	–	240.0.0.0–255.255.255.255	–

5.2 Direcciones IP reservadas (privadas)

- A → `10.0.0.0`
- B → `172.16.0.0 – 172.31.0.0`
- C → `192.168.0.0 – 192.168.255.0`

5.3 Máscara de subred

Permite determinar si una dirección IP pertenece a la misma red o no.
Ejemplo:

```
Dirección: 148.120.33.110
Máscara:   255.255.0.0
Red:       148.120.0.0
```

5.4 Protocolo IP

- No orientado a conexión.
- No fiable.
- Define el **datagrama IP**.

5.5 Notación CIDR

Ejemplo: `192.168.0.4/24`
Permite **subredes más flexibles** y eficientes que el sistema por clases.

5.6 IPv6

- Direcciones de **128 bits**.
- Mejora seguridad, autoconfiguración y rendimiento.

◇ 6. Capa de transporte

Gestiona la **comunicación entre procesos** dentro de los hosts.

6.1 Puertos

Servicio	Puerto
HTTP	80
HTTPS	443
FTP	21
DNS	53
SMTP	25
SSH	22

6.2 Protocolo TCP

- **Orientado a conexión**.
- **Fiable y ordenado.**
- Usa **ACKs y retransmisiones**.
- Comunicación **full-dúplex**.

6.3 Protocolo UDP

- **No orientado a conexión.**
- **No fiable.**
- Comunicación rápida (DNS, streaming).

◇ 7. Capa de aplicación

Proporciona la **interfaz entre el usuario y la red**.

Protocolo	Función
HTTP / HTTPS	Navegación web
SMTP / POP3 / IMAP	Correo electrónico
FTP	Transferencia de archivos
DNS	Resolución de nombres

La capa de aplicación **no son las apps**, sino **los protocolos que usan** (ej. WhatsApp usa un protocolo sobre TCP/IP).

🧩 Resumen general por capas TCP/IP

Capa	Función principal	Protocolos comunes
Aplicación	Servicios al usuario	HTTP, FTP, SMTP, DNS
Transporte	Comunicación entre procesos	TCP, UDP
Interred	Direccionamiento y enrutamiento	IP, ICMP, ARP
Enlace	Transmisión física y control de errores	Ethernet, Wi-Fi

---

💡 Conceptos clave

- OSI = 7 capas (teórico).
- TCP/IP = 4 capas (práctico).
- IP identifica host, **puerto** identifica aplicación.
- **TCP** = fiable, **UDP** = rápido.
- **CIDR** mejora la asignación de IPs.
- **IPv6** amplía el espacio de direcciones.

🌓 Unidad 6 – Seguridad Informática (Resumen Extendido – FADENA 2025)

Materia: Sistemas Operativos II
Carrera: Licenciatura en Ciberdefensa
Fuente: Material oficial FADENA – SO2 – Unidad 6 – Semana 12 (2025)

---

1. Seguridad de la Información

La **seguridad de la información** abarca todas las medidas preventivas y reactivas para proteger los datos —sin importar su formato o soporte— de accesos no autorizados, alteraciones indebidas o pérdida de disponibilidad.

Incluye políticas, procedimientos y controles que garantizan el resguardo de la información, tanto física como digital.

La información puede existir en distintos medios:

- Electrónicos (bases de datos, redes)
- Verbales (comunicaciones orales)
- Escritos o impresos (documentos físicos)

El objetivo central es preservar tres propiedades esenciales, conocidas como la **Tríada C-I-D**:

Propiedad	Qué protege	Pregunta clave	Ejemplo	Amenazas comunes	Medidas preventivas
Confidencialidad	Acceso restringido solo a usuarios autorizados	¿Quién puede acceder?	Historial médico, datos bancarios	Acceso no autorizado, ingeniería social, robo de dispositivos	Cifrado, contraseñas seguras, MFA, control de accesos
Integridad	Precisión y confiabilidad de los datos	¿Es esta información confiable?	Registros contables o diagnósticos médicos	Alteraciones maliciosas, errores humanos	Hash, firmas digitales, control de versiones
Disponibilidad	Accesibilidad de la información y sistemas cuando se necesitan	¿Puedo acceder cuando la necesito?	Acceso al correo o sistemas de emergencias	Ataques DDoS, fallos de hardware, cortes eléctricos	Backups, redundancia, planes de recuperación

◇ **Ejemplo práctico:**
En un entorno hospitalario digital:

- La **confidencialidad** asegura que solo el médico vea los historiales.
- La **integridad** mantiene los diagnósticos sin alteraciones.
- La **disponibilidad** garantiza que los sistemas estén activos durante emergencias.

🌍 Normas y Estándares Internacionales

La seguridad de la información se ha formalizado mediante la familia de normas **ISO/IEC 27000**, que define un marco de gestión para proteger datos sensibles:

Norma	Enfoque	Aplicación
27001	Sistema de Gestión de Seguridad de la Información (SGSI)	Única certificable

Norma	Enfoque	Aplicación
27002	Buenas prácticas y controles técnicos	Apoyo a la 27001
27005	Gestión de riesgos de seguridad	Evaluación de amenazas
27701	Privacidad y protección de datos personales	Extensión del SGSI
27032	Ciberseguridad	Enfoque en entornos digitales

Estas normas ayudan a mantener la **Confidencialidad, Integridad y Disponibilidad (CID)** bajo un marco internacional.

---

2. Seguridad Informática

Es la **rama de la seguridad de la información** que protege los datos cuando están **digitalizados** o **procesados en sistemas informáticos**.

Su objetivo es asegurar la **CID** de la información almacenada o transmitida electrónicamente.

Incluye aspectos técnicos como:

- Cifrado de datos.
- Autenticación de usuarios.
- Control de accesos.
- Protección frente a malware.
- Seguridad en redes y comunicaciones.

Ejemplo: un banco utiliza **doble factor de autenticación (MFA)** para validar accesos.

---

3. Software Maligno (Malware)

El **malware** (malicious software) es cualquier programa diseñado para dañar, alterar, espiar o robar información de un sistema sin consentimiento del usuario.

Tipo	Descripción	Ejemplo
Virus	Se adhiere a archivos y se propaga al ejecutarlos.	Archivos infectados en pendrives.
Gusano (Worm)	Se replica automáticamente por la red sin intervención humana.	“ILOVEYOU”.
Troyano (Trojan)	Se disfraza de programa legítimo para dar acceso remoto al atacante.	Crack o instalador falso.
Spyware	Espía al usuario, recolecta datos personales o bancarios.	Roba credenciales del navegador.
Adware	Muestra anuncios no deseados o rastrea hábitos de navegación.	Pop-ups publicitarios invasivos.

Tipo	Descripción	Ejemplo
Ransomware	Cifra archivos y pide rescate en criptomonedas.	“WannaCry”.
Rootkit	Oculta procesos o malware en el sistema.	Desactiva antivirus o altera el kernel.
Keylogger	Registra pulsaciones del teclado.	Robo de contraseñas.
Botnet	Red de equipos infectados controlados remotamente.	Ataques DDoS.

Prevención recomendada (según FADENA):

- Antivirus actualizado y monitoreo activo.
- Control de accesos y privilegios.
- Educación sobre phishing e ingeniería social.
- Políticas de backup y actualización de sistemas.

---

4. Encriptación o Cifrado de Datos

El **cifrado** transforma datos legibles (**texto plano**) en ilegibles (**texto cifrado**) mediante un algoritmo y una clave.

Fórmulas básicas:

- Cifrado: `C = E(P, KE)`
- Descifrado: `P = D(C, KD)`

Principio de Kerckhoffs:
La seguridad no debe depender del secreto del algoritmo, sino de la clave utilizada.

Breve Historia de la Criptografía

- **Antigüedad:** jeroglíficos egipcios, Cifrado César, Scytale espartana.
- **Edad Media:** cifrado polialfabético de Alberti, cuadrado de Vigenère.
- **Siglo XX:** Enigma, DES, 3DES, RSA.
- **Actualidad:** AES, ECC, PGP.

Cifrado Simétrico

Usa **una única clave** compartida para cifrar y descifrar.

Funcionamiento:

1. El emisor cifra con la clave compartida.
2. El receptor descifra con la misma clave.

Ventajas	Desventajas
Rápido y eficiente.	Requiere canal seguro para compartir la clave.

Ventajas	Desventajas
Ideal para grandes volúmenes de datos.	Si la clave se filtra, todo el sistema queda comprometido.
Utilizado en VPN y discos cifrados.	No asegura prueba de origen ni no repudio.

Ejemplos: DES, 3DES, AES, RC4, Blowfish.

🔑 Cifrado Asimétrico

Usa **dos claves**:

- **Pública** (difundida libremente).
- **Privada** (solo el propietario la posee).

Casos de uso:

1. **Confidencialidad:** el emisor usa la clave pública del receptor.
2. **Prueba de origen:** el emisor cifra con su clave privada.
3. **Ambas (signed):** combina confidencialidad y autenticación.

Ventajas	Desventajas
Garantiza confidencialidad y autenticidad.	Más lento que el simétrico.
Mejora la gestión de claves.	Ineficiente para grandes volúmenes.

Ejemplos: RSA, ECC, DSA, ElGamal.

🎭 Esteganografía

Oculta la existencia del mensaje dentro de otros medios (imagen, audio o texto).
A diferencia del cifrado, **no transforma el contenido**, sino que lo **disimula**.

Ejemplo: ocultar datos modificando los bits menos significativos de una imagen (LSB).

---

5. Red Privada Virtual (VPN)

Una **VPN (Virtual Private Network)** extiende una red privada sobre Internet mediante **túneles cifrados**, protegiendo la comunicación entre el usuario y la red interna.

Usos:

- Acceso remoto seguro a redes corporativas.
- Cifrado de tráfico en conexiones públicas.
- Ahorro de costos frente a líneas dedicadas.

Beneficios clave (FADENA):

- Confidencialidad del tráfico.

- Integridad de los datos.
- Acceso remoto flexible desde cualquier ubicación.

---

6. Firmas y Certificados Digitales

✍️ Firma Digital

Conjunto de datos electrónicos asociados a un documento digital que:

- Identifican al firmante.
- Garantizan la **integridad** del documento.
- Aseguran el **no repudio** (el autor no puede negar haber firmado).

Aplicaciones comunes:

- Firma de correos electrónicos.
- Presentación de trámites en línea.
- Declaraciones fiscales o informes digitales.

📄 Certificado Digital

Documento emitido por una **Autoridad de Certificación (CA)** que asocia la identidad de una persona con su clave pública.

Ejemplo práctico:
En el navegador, al hacer clic en el candado del sitio `https://campus.fadena.undef.edu.ar`, se puede visualizar el certificado digital que garantiza la autenticidad del dominio.

⚙️ Proceso de Firma Digital

1. **Generación del hash:** se calcula una “huella digital” única del documento.
2. **Cifrado del hash:** se cifra con la clave privada del firmante.
3. **Creación de la firma:** el hash cifrado se adjunta al documento.

Si el documento se altera, el hash resultante no coincide, invalidando la firma.

---

7. Autenticación

Proceso mediante el cual un sistema o persona **demuestra su identidad** ante otro sistema.

Partes involucradas:

- **Remitente:** quien intenta autenticarse.
- **Verificador:** quien valida la identidad.

Importancia:
Evita accesos indebidos y garantiza control sobre recursos críticos.

🔢 Métodos de Autenticación

Tipo	Qué representa	Ejemplo
Conocimiento	Algo que sabes	Contraseña, PIN, pregunta secreta
Posesión	Algo que tienes	Token, tarjeta, SMS con código
Característica Física (biometría)	Algo que eres	Huella, rostro, iris

Autenticación Multifactor (MFA):
Combina dos o más de los factores anteriores para incrementar la seguridad (ejemplo: contraseña + huella).

🔗 Proceso de Conexión del Usuario

1. **Conexión Inicial (Puesto de trabajo):** el usuario se autentica al iniciar sesión y obtiene acceso a recursos básicos (archivos, red, impresoras).
2. **Conexión a una aplicación específica:** se realiza una autenticación adicional dentro del software para validar permisos específicos.

Una gestión correcta de **identidades y accesos (IAM)** garantiza que cada usuario tenga los derechos adecuados según su rol.

🔚 Conclusión

La **seguridad informática** es un pilar esencial de la ciberdefensa moderna.
Su misión es mantener la **Confidencialidad, Integridad y Disponibilidad** de la información frente a amenazas tecnológicas.

Resumen clave:

- **CID:** base de toda política de seguridad.
- **Malware:** principal amenaza.
- **Cifrado:** herramienta de defensa.
- **Firmas y certificados:** garantizan identidad y autenticidad.
- **VPN:** protección del canal de comunicación.
- **Autenticación:** control de acceso y confianza.

Licenciatura en Ciberdefensa

Sistemas Operativos II

Unidad 6 – Semana 13 (Año 2025)

Contenidos

- Firewalls

- IDS/IPS
- Seguridad en sistemas operativos
- Gestión de usuarios y credenciales
- Políticas de cuentas y contraseñas seguras
- Administración de grupos, roles y permisos

---

1. Cortafuegos (Firewalls)

Un **firewall** es un sistema (hardware o software) que **controla y filtra el tráfico de red** según las políticas de seguridad de una organización.
Se ubica entre la **red interna** y **externa (Internet)**, actuando como una **barrera** que protege de accesos no autorizados.

1.1 Tipos de Firewalls

1.1.1 Firewall de Filtrado de Paquetes (Stateless)

- Analiza los **campos de los paquetes IP** (IP origen/destino, puerto, MAC).
- Evalúa cada paquete **de forma independiente**, sin recordar conexiones previas.
- Requiere **reglas separadas** para tráfico entrante y saliente.
- Ineficiente ante conexiones bidireccionales.

1.1.2 Firewall de Inspección con Estado (Stateful Inspection)

- Mantiene una **tabla de estados** con las conexiones activas.
- Analiza el contexto de cada paquete.
- **Ventajas:** mayor seguridad, detección de ataques, filtrado contextual.
- **Desventajas:** consumo de recursos y configuración más compleja.
- Solo necesita **una regla de salida**: el retorno se valida automáticamente.

1.1.3 Firewall a Nivel de Circuito (Circuit-level Gateway)

- Opera en la **capa de transporte**.
- Valida la conexión inicial y permite el flujo posterior con mínima inspección.
- Riesgo: puede dejar pasar datos maliciosos una vez validado el circuito.

1.1.4 Firewall a Nivel de Aplicación (Application Level – Proxy)

- Opera en la **capa de aplicación**, actuando como **intermediario**.
- **Inspección profunda del tráfico (payload)**.
- **Oculta IP internas** y **rompe la conexión directa** entre cliente y servidor.
- Alta seguridad, aunque introduce **latencia**.

1.1.5 Cortafuegos Personal (Host-Based Firewall)

- Instalado directamente en un equipo.
- Controla el tráfico **entrante y saliente del host**.
- Ejemplos: **Firewall de Windows Defender**, **iptables (Linux)**.

- Ideal para entornos móviles o fuera del perímetro corporativo.

---

2. Detección y Prevención de Intrusiones (IDS/IPS)

La seguridad moderna combina **firewalls, IDS/IPS y SIEM** para crear perímetros inteligentes y adaptativos.

2.1 IDS (Intrusion Detection System)

- Monitorea tráfico o actividad de sistemas para detectar **patrones sospechosos**.
- Función **pasiva**: genera alertas, no bloquea tráfico.
- **Tipos:**
  - **NIDS:** supervisa la red completa; limitado con tráfico cifrado.
  - **HIDS:** instalado en un host; detecta ataques internos, consume recursos.

2.2 IPS (Intrusion Prevention System)

- Evolución activa del IDS.
- Se instala **en línea**, analiza y bloquea amenazas en tiempo real.
- **Acciones:** bloquear tráfico, reiniciar conexiones, poner IPs en cuarentena o engañar atacantes.
- **Riesgos:** falsos positivos y latencia.

2.3 Diferencias entre IDS e IPS

Característica	IDS (Detección)	IPS (Prevención)
Naturaleza	Pasiva	Activa
Acción	Alerta	Bloquea/Mitiga
Ubicación	Fuera de línea	En línea
Tiempo de respuesta	Manual	Automático

2.4 Ejemplos Actuales

Tipo	Herramienta	Clasificación	Características
IDS	Snort	NIDS	Código abierto, basado en firmas
IDS	Suricata	NIDS/HIDS	Multihilo, DPI, compatible con Snort
IDS	Zeek (Bro)	NIDS	Análisis contextual, scripting forense
IDS	OSSEC	HIDS	Monitoreo de logs y archivos
IPS	Cisco Firepower	IPS/NGFW	Integración con SecureX
IPS	Fortinet FortiIPS	IPS dedicado	Protección contra exploits
IPS	Palo Alto Threat Prevention	IPS integrado	Análisis en tiempo real
IPS	Trend Micro TippingPoint	IPS dedicado	Defensa contra vulnerabilidades

---

3. Gestión de Usuarios y Grupos en Linux

La gestión de usuarios es fundamental para la **seguridad, trazabilidad y estabilidad** del sistema.

3.1 Necesidad de Usuarios

1. **Seguridad (Principio de Mínimo Privilegio)**: limita permisos a lo necesario.
2. **Responsabilidad/Auditoría**: cada acción es atribuible a un usuario.
3. **Estabilidad/Personalización**: cada usuario tiene su propio entorno aislado.

3.2 Fundamentos

- Cada usuario tiene un **UID único** y un **directorio personal (/home)**.
- Los usuarios comunes operan con privilegios limitados.
- En Linux, el administrador utiliza **root** (UID 0).

3.3 Tipos de Usuarios

Tipo	UID	Interactivo	Función
Superusuario (root)	0	Sí	Control total del sistema
Usuarios del sistema	1–999	No	Ejecutan servicios (daemons)
Usuarios normales	≥1000	Sí	Uso diario y personal

3.4 Grupos (group)

- Permiten **asignar permisos colectivos**.
- Cada usuario pertenece a un **grupo primario** y puede estar en varios **secundarios**.
- **UPG (User Private Group):** grupo primario con el mismo nombre que el usuario.

3.5 Atributos de Usuario

- **UID y nombre de cuenta.**
- **Grupos asociados (primarios/secundarios).**
- **Shell asignada (bash).**
- **Contraseña y caducidad.**
- **Cuotas de disco y auditoría.**

---

4. Políticas de Cuentas y Contraseñas Seguras

4.1 Políticas de Usuarios

Definen la **creación, modificación y eliminación** de cuentas.
Se basan en las normas **ISO/IEC 27001 y 27002 (Control de Acceso A.9)**.

Controles clave:

- Registro formal y gestión de accesos.
- Asignación por rol.

- Gestión de privilegios especiales.
- Revisión y eliminación de accesos caducos.
- Seguridad de credenciales.

4.2 Políticas de Contraseñas

- Mecanismo principal de autenticación.
- Basadas en la **ISO/IEC 27002**.

Buenas prácticas:

- Longitud mínima: **12 caracteres**.
- Mezclar letras, números y símbolos.
- Evitar datos personales.
- Cambiar al detectar compromiso.
- Bloquear tras intentos fallidos.
- No reutilizar contraseñas previas.
- Almacenamiento **cifrado**.
- Uso recomendado de **MFA (autenticación multifactor)**.

---

5. Roles y Permisos

Conceptos Clave

Término	Definición	Ejemplo
Recurso	Objeto protegido	Archivo, directorio, impresora
Permiso	Acción permitida	Leer, escribir, ejecutar
Identidad	Usuario o servicio	“jlopez”, “apache”
Rol	Conjunto de permisos	Administrador, usuario, invitado

5.1 Modelos de Control de Acceso

5.1.1 DAC (Discretionary Access Control)

- Controlado por el **propietario del recurso**.
- Flexible pero menos seguro (errores humanos posibles).
- Implementado con `chmod` en Linux o **ACLs en Windows**.

5.1.2 MAC (Mandatory Access Control)

- Controlado por el **administrador o sistema operativo**.
- Basado en **etiquetas de seguridad** (“Secreto”, “Confidencial”).
- Ejemplo: **SELinux**, **AppArmor**.
- Muy seguro pero rígido.

5.1.3 RBAC (Role-Based Access Control)

- Control basado en **roles predefinidos** (ej. “Contador”, “Gerente”).
- Escalable y usado en entornos corporativos.
- Implementado en **Active Directory**, **LDAP**, etc.

Característica	DAC	MAC	RBAC
Poder de decisión	Propietario	Administrador	Rol
Rigidez	Baja	Alta	Media
Principio de Mínimo Privilegio	Depende del usuario	Inherente	Administrado
Uso típico	PCs personales	Militar/gubernamental	Empresas

Conclusión Unidad 6

La **Unidad 6** integra los pilares de la **seguridad operativa en sistemas Linux**, abordando desde la defensa perimetral (firewalls, IDS/IPS) hasta la **gestión interna de usuarios, contraseñas y permisos**.
Su correcta implementación garantiza la **confidencialidad, integridad y disponibilidad** de los recursos informáticos en entornos corporativos o de defensa.
